

Lab 1 - Advanced MPI

Large Scale Hybrid Computing

University of Minho

The Lab 1 focus on the development of parallel code using MPI's one sided asynchronous communication primitives. Use a Deucalion node with ARM CPUs (by specifying the partition **normal-arm** when submitting a job). Compile the code either in the frontend or on the compute nodes (advised).

This lab tutorial includes five exercises to be solved during the lab class. Copy the source files, as usual, from `/projects/F202500010HPCVLABUMINHO/LSHC/lab1` to your home directory in Deucalion:

```
cp -r /projects/F202500010HPCVLABUMINHO/LSHC/lab1 .
```

To load the compiler in the environment use the following commands (do not forget to also use them in your submission script):

GNU Compiler: `module load GCC/14.3.0.`

MPI Libraries: `module load OpenMPI/5.0.8-GCC-14.3.0.`

You can use other GNU/OpenMPI combinations at your own risk, but ensure that OpenMPI is compiled with the correct version of the compilation toolchain.

1. Analyse the codebase

Goals: to understand the computation and communication requirements of a medium-complexity stencil code.

The code simulates 2D heat diffusion, a classic proxy for many high-performance scientific applications ranging from weather forecasting to seismic analysis. In this scenario, a large square surface is initialised with a cold temperature everywhere except for a constant heat source at the centre. To simulate the spread of thermal energy over time, the simulation uses a "5-point stencil" operation, where the temperature of any given point at the next time step is calculated as the average of its four immediate neighbours.

Parallelisation is achieved through domain decomposition, in which the global grid is split into smaller tiles assigned to different MPI processes. This creates a boundary dependency: calculating the temperature for the edge cells of a local tile requires data owned by a neighbouring process. To resolve this, each process maintains a "halo" (or "ghost zone"), a ring of extra memory used to store copies of its neighbours' boundary data. Consequently, this code can be

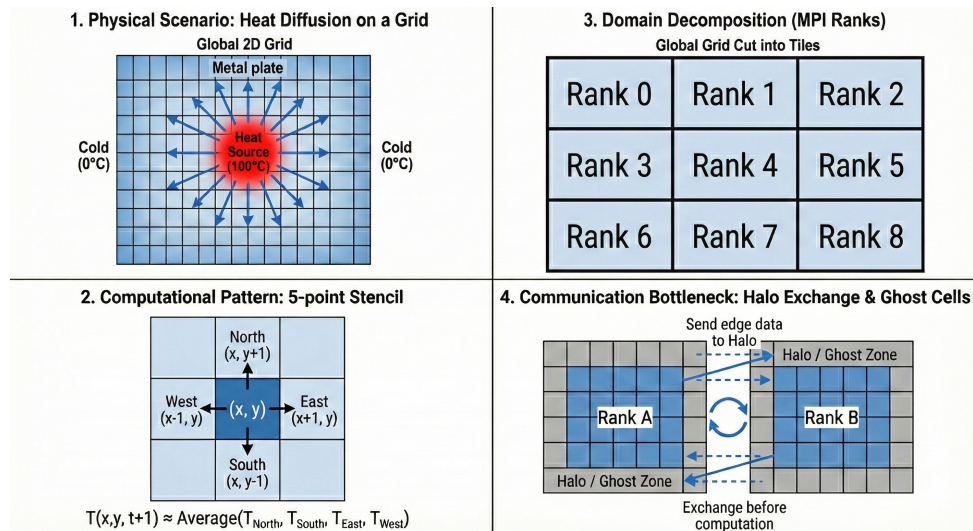


Figure 1: Visualisation of the data decomposition, parallelisation, and communication scheme.

used as a benchmark for nearest-neighbour communication efficiency, testing how quickly the system can synchronise these boundary layers across the network.

Lab 1.1 Execute the sequential version of the code and visualize the results with the provided script. Record the execution time for various grid sizes that you see fit testing in the following exercises. Does the execution time increase linearly with the size of the grid?

Lab 1.2 Visualise the initial and processed grid using the provided Python script. While not a rigorous test, this can be used to assess if the parallel code provides the same result as the original sequential version.

How does varying the amount of steps in the simulation affect the results? Does the execution time increase linearly with $\#steps$?

2. MPI with Memory Windows

Goals: to develop skills in developing code with MPI's Window one-sided communication primitives.

Lab 2.1 Consider the provided code skeleton. Complete the code sections highlighted in the file with the primitives adequate for each phase of the communication:

Remote memory allocation: `MPI_Win_allocate`

Halo datatype definition: `MPI_Type_vector` and `MPI_Type_commit`

Synchronisation: `MPI_Barrier` and `MPI_Win_fence`

Remote memory copy: `MPI_Put`

Ensure that the code provides an accurate solution when compared to its sequential version.

Lab 2.2 Measure and compare the performance of your MPI version with the sequential code for the datasets chosen in 1.1 using a single node. Using the larger dataset, identify a #processes adequate to achieve the best execution time for 1, 2 and 4 nodes, and perform those measurements.

Does the execution time scale linearly with the problem size for the single node? Does the execution time reduces in proportion to the number of nodes used? If not, speculate about the possible bottlenecks (educated guesses only!).

Lab 2.3 Parallelise the computation that each process does using OpenMP. Identify possible loops or tasks to run concurrently and code the required OpenMP pragma directives to ensure their parallel and efficient execution.

Measure and compare the performance versus the MPI only code using combinations of #processes and #threads per process most adequate to the A64FX (ask for help to understand how processes/threads should match the PU architecture). Measure the execution time for a large grid size using one and 4 A64FX-based nodes and identify the best #process-#threads combination.